# Memory Management and Namespaces

**COMP315**

**2023**

UNIVERSITY OF KWAZULU-NATAL

# Memory Management

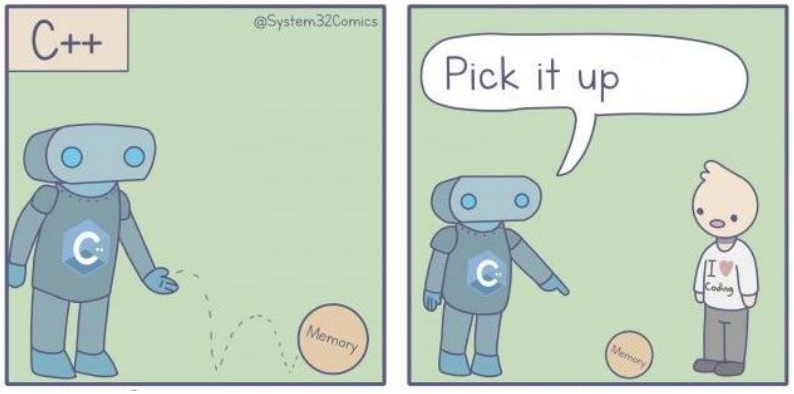
**COMP315**

**2023**

UNIVERSITY OF KWAZULU-NATAL

# Memory Management

- As you know, computers have memory. Memory stores pieces of **data**
- When we code, we **store** our **data in memory**
- From a Java perspective, memory allocation is abstracted from us. Java has a **garbage collector**. The garbage collector **discards** any data that we do not use anymore. In other words, the garbage collector is smart enough to realise that a dynamic variable, that we created, is **no longer in use**. It then goes to the memory address that stores that variable's data, and it **frees up the memory** by **deleting the data**
- C++ does **not** have a garbage collector, and so we need to **manually** clear memory once a **dynamic variable goes out of scope**



# What Is Dynamic Memory in C++?

- **Dynamic Memory** refers to the programmer **manually** allocating and deallocating memory
- Dynamically allocated memory is allocated on **Heap** and non-static and local variables get memory allocated on **Stack**
- There are benefits to dynamic memory. This is actually one of the reasons that makes C++ favourable to many programmers as it allows us the flexibility to allocate and deallocate memory whenever we need to
- If dynamic memory is handled well, C++ can be a very effective language in optimising performance
- Hence, dynamic memory is what makes C++ both famous and infamous amongst the programming community

UNIVERSITY OF KWAZULU-NATAL

# What Are Dynamic Variables in C++?

- For variables such as `int a` or `char str[10]` memory is **automatically allocated and deallocated**
- For dynamically allocated memory, for example, `int *p = new int[10]`, it is the **programmers responsibility to deallocate memory when no longer needed**. If the programmer does not deallocate memory, it causes a **memory leak** (meaning that memory is not deallocated until program **terminates**)
- Dynamic variables are allocated memory at **run time**. C++ has two operators `new` and `delete` that perform the task of **allocating** and **freeing** the memory

UNIVERSITY OF KWAZULU-NATAL

# The `new` Operator

- The **`new`** operator denotes a request for memory allocation on the **Heap**. If sufficient memory is available, `new` operator **initializes (allocates)** the memory and returns the **address** of the newly allocated and initialized memory to the pointer variable
- To allocate memory of any data type, the syntax is:

```
pointer_variable_name = new data_type;
```

  - *data-type could be any built-in data type including array or any user defined data types including structure and class.*

- Example:

```
int *p = new int;
```

UNIVERSITY OF KWAZULU-NATAL

# The `delete` Operator

- The `delete` operator **deallocates** dynamically allocated memory
- To deallocate memory of any data type, the syntax is:

```
delete pointer_variable_name;
```

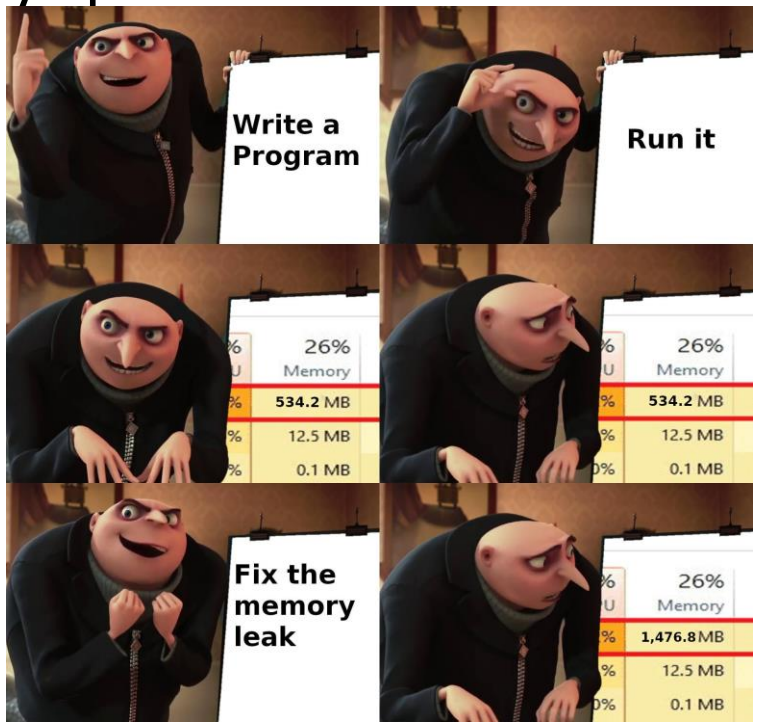
  - *The data-type does not need to be specified. Only the name of the variable needs to follow the keyword* ***delete***
- Example:

```
delete p;
int *q = new int;
delete q;
```

UNIVERSITY OF KWAZULU-NATAL

# The period of time that a variable takes up space in memory

- The period of time during which a variable has memory space associated with it:
  - **begins** when space is **allocated**
  - **ends** when space is **de-allocated**




- Three categories of variables in memory storage:
  - Static - start to end of program execution
  - Automatic (Stack) - start to end of declaring function's execution
  - Heap - starts with "new“ and ends when the "delete" statement is executed

UNIVERSITY OF KWAZULU-NATAL

# Deep And Shallow Copies

- A **shallow copy** copies only the class data members, and does not copy any pointed-to data

- a **deep copy** copies not only the class data members, but also makes separately stored copies of any pointed-to data

UNIVERSITY OF KWAZULU-NATAL

# Deep Copy And Shallow Copy Example:

## Example 1

```
int b = 5;
int a = b;
```

## Example 2

```
int *b = new int;
*b = 5;
int* a = b;
```

C++ performs **Shallow Copy** by default. So in both these examples, the copy we see is shallow in nature. In **Example 1**, *a* is assigned the value of *b*. In this case, *a* and *b* are independent variables, i.e. modifying *a* will not affect *b* and vice-versa. This works perfectly fine when the variables are stack allocated.

However in **Example 2**, both *a* and *b*, although independent pointers, point to the same heap memory location. Modifying one will affect the other. If the copy had been a **deep copy** instead, a new heap memory would have been allocated for *a*.

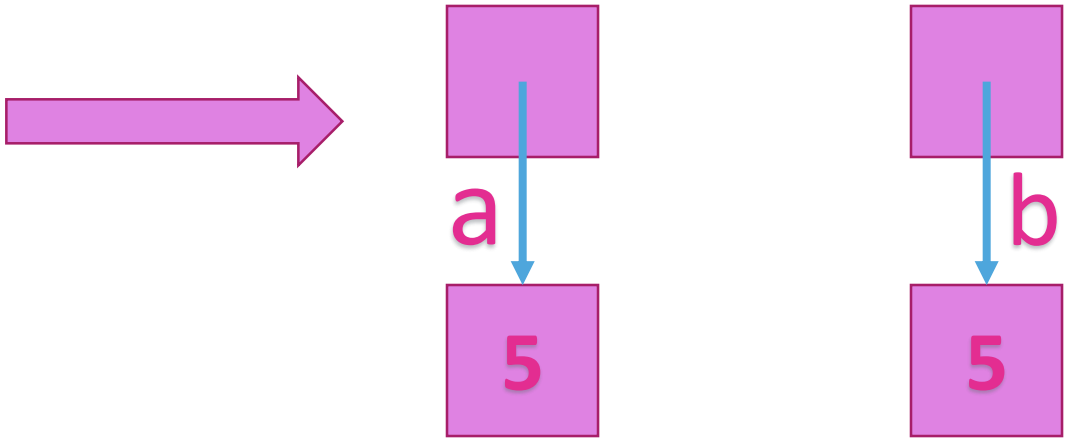


UNIVERSITY OF KWAZULU-NATAL

# Deep Copy And Shallow Copy Example: Graphical Explanation

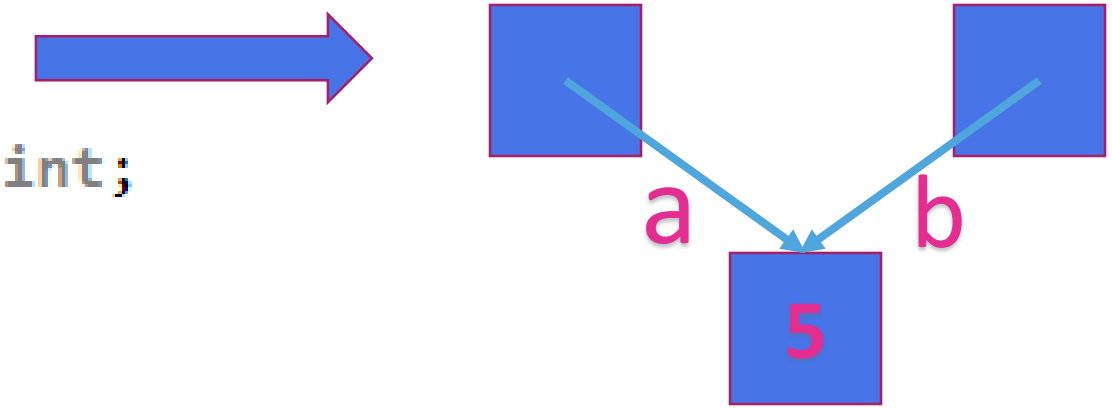
**Example 1**

```
int b = 5;
int a = b;
```




**Example 2**

```
int *b = new int;
*b = 5;
int* a = b;
```






UNIVERSITY OF KWAZULU-NATAL

# Namespaces

**COMP315**

**2023**

UNIVERSITY OF KWAZULU-NATAL

# Namespaces Management

- A **namespace** is a collection of name definitions, such as class definitions and variable declarations
  - If a program uses classes and functions written by different programmers, it may be that the same name is used for different things.
  - Namespaces help us deal with this problem.

UNIVERSITY OF KWAZULU-NATAL

# The Using Directive

- `#include <iostream>` places names such as `cin` and `cout` in the `std` namespace.

- The program does not know about names in the `std` namespace until you add

```
using namespace std;
```

(if you do not use the std namespace, you can define cin and cout to behave differently)

UNIVERSITY OF KWAZULU-NATAL

# The Global Namespace

- Code that you write is in a namespace
  - It is in the **global namespace** unless you specify a namespace.
  - The global namespace does not require the using directive.

UNIVERSITY OF KWAZULU-NATAL

# Name Conflicts

- If the same name is used in two namespaces
  - The namespaces cannot be used at the same time
  - Example: If my_function is defined in namespaces ns1 and ns2, the two versions of my_function could be used in one program by using local using directives this way:

```
{
  using namespace ns1;
  my_function( );
}
```

```
{
  using namespace ns2;
  my_function( );
}
```

UNIVERSITY OF KWAZULU-NATAL

# Scope Rules For using

- A block is a list of statements enclosed in `{ }`s.
- The scope of a **`using`** directive is the block in which it appears.
- A `using` directive placed at the beginning of a file, outside any block, applies to the entire file.

UNIVERSITY OF KWAZULU-NATAL

# Creating a Namespace

- To place code in a namespace
  - Use a **namespace grouping**
    - ```
      namespace Name_Space_Name
      {
              Some_Code
      }
      ```

- To use the namespace created
  - Use the appropriate using directive
    - ```
      using namespace Name_Space_Name;
      ```

UNIVERSITY OF KWAZULU-NATAL

# Namespaces: Declaring a Function

- To add a function to a namespace
  - Declare the function in a namespace grouping

```
namespace savitch1
{
        void greeting( );
}
```

UNIVERSITY OF KWAZULU-NATAL

# Namespaces: Defining a Function

- To define a function declared in a namespace
  - Define the function in a namespace grouping

```
namespace savitch1
{
    void greeting( )
      {
          cout << "Hello from namespace savitch1.\n";
      }
}
```

UNIVERSITY OF KWAZULU-NATAL

# Namespaces: Using a Function

- To use a function defined in a namespace
  - Include the using directive in the program where the namespace is to be used
  - Call the function as the function would normally be called

```
int main( )
{
        using namespace savitch1;
        greeting( );
}
```

UNIVERSITY OF KWAZULU-NATAL

# A Namespace Problem

- Suppose you have the namespaces below:

```
namespace ns1
{
        fun1( );
        my_function( );
 }
```

```
namespace ns2
{
        fun2( );
        my_function( );
}
```

- Is there an easier way to use both namespaces considering that `my_function` is in both?

UNIVERSITY OF KWAZULU-NATAL

# Qualifying Names

- **Using declarations** (not directives) allow us to select individual functions to use from namespaces

  - **using ns1::fun1;** //makes **only** fun1 in ns1 available
    - The scope resolution operator identifies a namespace here
    - Means we are using <u>only</u> namespace ns1's version of fun1

  - If you only want to use the function once, call it like this

    **ns1::fun1( );**

UNIVERSITY OF KWAZULU-NATAL

# Qualifiying Parameter Names

- To qualify the type of a parameter with a using declaration
  - Use the namespace and the type name

```
int get_number (std::istream input_stream)
    …
```

    - istream is the istream defined in namespace std
    - If istream is the only name needed from namespace std, then you do not need to use

```
using namespace std;
```

UNIVERSITY OF KWAZULU-NATAL

# Directive/Declaration (Optional)

- A using declaration (`using std::cout`;) makes only one name available from the namespace.

- A using directive makes all the names in the namespace available

UNIVERSITY OF KWAZULU-NATAL

# A Subtle Point (Optional)

- A using directive potentially introduces a name
- If ns1 and ns2 both define my_function,

```
using namespace ns1;
using namespace ns2;
```

is fine, provided my_function is never used!

UNIVERSITY OF KWAZULU-NATAL

# Unnamed Namespaces

- As we have done helper functions so far, they are not really hidden  We would like them to be local to the implementation file to implement information hiding.

- The **unnamed namespace** can hide helper functions
  - Names defined in the unnamed namespace are local to the **compilation unit**
    - A compilation unit is a file (such as an implementation file) plus any file(s) #included in the file

UNIVERSITY OF KWAZULU-NATAL

# The unnamed grouping

- Every compilation unit has an unnamed namespace
  - The namespace grouping is written as any other namespace, but no name is given:

```
namespace
{
    void sample_function( )
     …
}   //unnamed namespace
```

UNIVERSITY OF KWAZULU-NATAL

# Compilation Units Overlap

- A header file is #included in two files
  - It is in two compilation units
  - Participates in two unnamed namespaces!
  - This is OK as long as each of the compilation units makes sense independent of the other
    - A name in the header file's unnamed namespace cannot be defined again in the unnamed namespace of the
      implementation or application file

UNIVERSITY OF KWAZULU-NATAL

# Naming Namespaces

- To avoid choosing a name for a namespace that has already been used
  - Add your last name to the name of the namespace
  - Or, use some other unique string

UNIVERSITY OF KWAZULU-NATAL

# Global or unnamed?

- Names in the global namespace have global scope (all files)
  - They are available without a qualifier to all the program files

- Names in the unnamed namespace are local to a compilation unit
  - They are available without a qualifier within the compilation unit

UNIVERSITY OF KWAZULU-NATAL